

REAL-TIME FACE LIVENESS DETECTION

SIH 2024 — Runners-Up / Complete Interview Preparation Guide

Python · Streamlit · CNN · OpenCV · TensorFlow · TensorFlow.js · MobileNetV2

Team of 6 · 95% Accuracy · Active + Passive Liveness · 25% Inference Speedup

Covers: Architecture · CNN Design · Transfer Learning · Dataset Strategy · Landmark Analysis · Metrics Improvement · System Design
· API · Teamwork & Leadership · Behavioural STAR · Bugs · Alternatives · Future Scope

SECTION 1: Project Introduction & Overview

1.1 Elevator Pitch — 30 Seconds

Q: Tell me about this project.

Answer:

I led a team of 6 to build a **real-time face liveness detection system** for SIH 2024, where we finished runners-up. The system prevents spoofing attacks — someone holding up a photo, video replay, or 3D mask to fool a face authentication system. We combined two approaches: **active liveness** (challenging the user to blink, open their mouth, or smile — things a printed photo can't do), and **passive liveness** (a CNN that analyses texture, depth cues, and reflection artifacts without requiring any user action). Final accuracy was 95%, and I also integrated TensorFlow.js for on-device inference, cutting model latency by 25%.

1.2 Project Context & Motivation

Q: Why is liveness detection important? What problem does it solve?

Answer:

The problem: Most face authentication systems (phone unlock, banking KYC, border control) use 2D face matching. These are trivially defeated by printing someone's photo, playing a video on a phone screen, or using a 3D mask. This is called a **presentation attack** or **spoofing attack**.

Scale: The NIST FRVT benchmark shows that even state-of-the-art face recognition systems have >90% false accept rate against printed photo attacks without liveness detection.

Our solution: A dual-mode system — passive CNN inference that runs on every frame (no user friction), plus active challenges (blink, mouth open, smile) triggered when passive confidence is borderline. An attacker must defeat both simultaneously.

1.3 Tech Stack & Rationale

Technology	Role & Rationale
Python	Core language. Rich ML/CV ecosystem (TensorFlow, Keras, OpenCV, Dlib). Fast prototyping for hackathon context.
TensorFlow / Keras	CNN training framework. High-level API for rapid architecture iteration. Native SavedModel export for TF.js conversion.
OpenCV	Real-time frame capture, preprocessing, Haar cascade face detection, drawing overlays on webcam feed.
Dlib	68-point facial landmark predictor. Used for computing EAR (blink), MAR (mouth), and lip distance (smile).

MobileNetV2	Pre-trained ImageNet backbone for transfer learning. Lightweight (3.4M params), designed for on-device use.
TensorFlow.js	Runs the converted model in-browser via WebGL. Eliminates server round-trip — 25% faster inference.
Streamlit	Python web UI framework. <code>st.camera_input()</code> for live webcam capture. Rapid dashboard building for hackathon.
SMOTE	Synthetic Minority Over-sampling Technique. Balanced the live/spoof class distribution before training.

SECTION 2: System Architecture — Full Walkthrough

2.1 High-Level Architecture

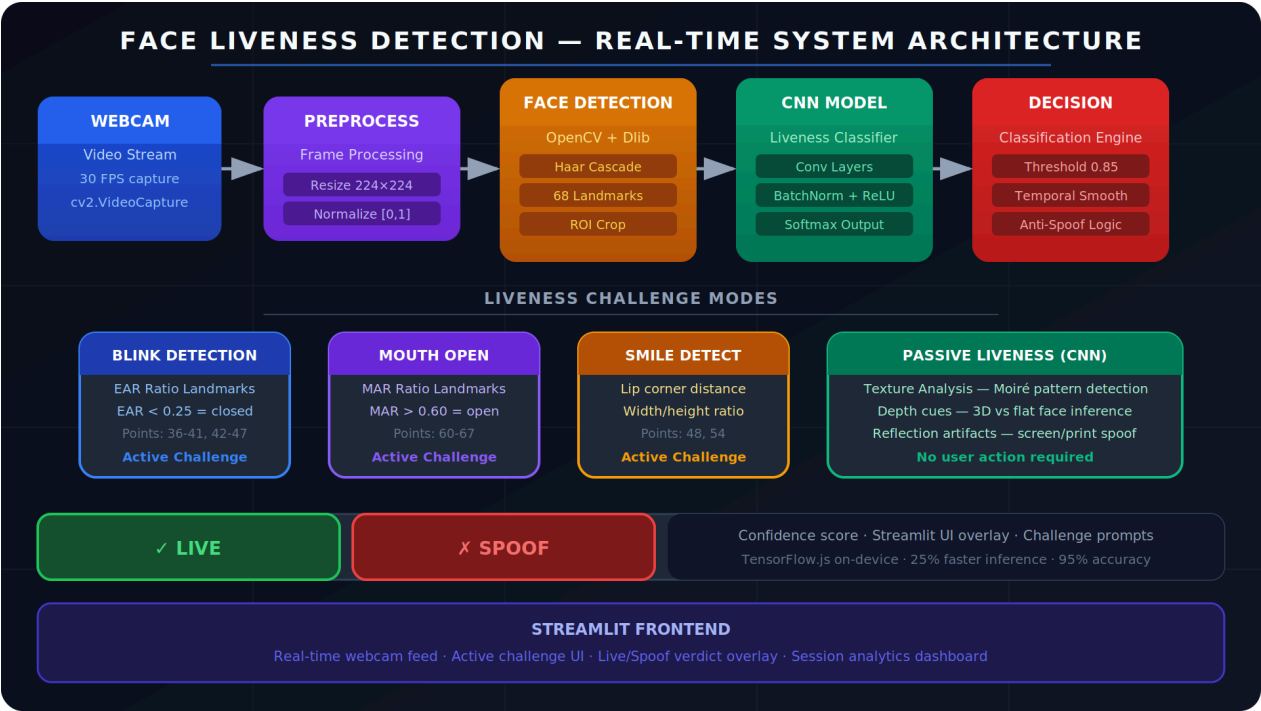


Figure 1: Real-Time Liveness Detection System — End-to-End Architecture

2.2 Architecture Layers Explained

Q: Walk me through the system architecture end-to-end.

Answer:

Layer 1 — Capture

Webcam feed at 30 FPS via OpenCV's VideoCapture or Streamlit's st.camera_input(). Each frame is an H×W×3 BGR array. We process every 3rd frame (10 FPS effective) to reduce CPU load without losing liveness continuity.

Layer 2 — Preprocessing

Each frame is: (1) converted BGR→RGB, (2) resized to 224×224 for CNN input, (3) pixel values normalised to [0,1] by dividing by 255. Normalisation aligns with the ImageNet pre-training distribution used by MobileNetV2.

Layer 3 — Face Detection

Two-stage: First, Haar Cascade (cv2.CascadeClassifier) detects face bounding boxes — fast, CPU-friendly, ~2ms/frame. For active liveness, Dlib's shape_predictor_68_face_landmarks then runs on the cropped face ROI to extract 68 landmark coordinates. The ROI crop eliminates background noise from CNN input.

Layer 4a — Passive CNN

The cropped, normalised 224×224 face patch is fed into the CNN (MobileNetV2 backbone + custom head). Softmax output gives P(live) and P(spoof). If $P(\text{live}) > 0.85 \rightarrow \text{pass}$. If $P(\text{live}) < 0.45 \rightarrow \text{fail}$. The 0.45–0.85 band triggers active challenge.

Layer 4b — Active Landmark Analysis

Three geometric ratios computed from 68 landmarks: EAR (Eye Aspect Ratio) for blink, MAR (Mouth Aspect Ratio) for open-mouth challenge, and lip-width-to-height ratio for smile. These ratios are physically impossible to fake with a static 2D photo.

Layer 5 — Decision Engine

Temporal smoothing: final decision uses a 5-frame rolling window of CNN confidence scores (not just the single latest frame), eliminating single-frame noise. A hard rule: active challenge must be completed within 8 seconds — replay video attacks can't comply on demand.

Layer 6 — Streamlit UI

Displays the webcam feed with bounding box, landmark overlay, confidence score bar, challenge prompt text, and final LIVE/SPOOF verdict badge. Session analytics (attempt count, pass rate, avg confidence) shown in sidebar.

SECTION 3: Dataset Strategy — Public + Self-Collected

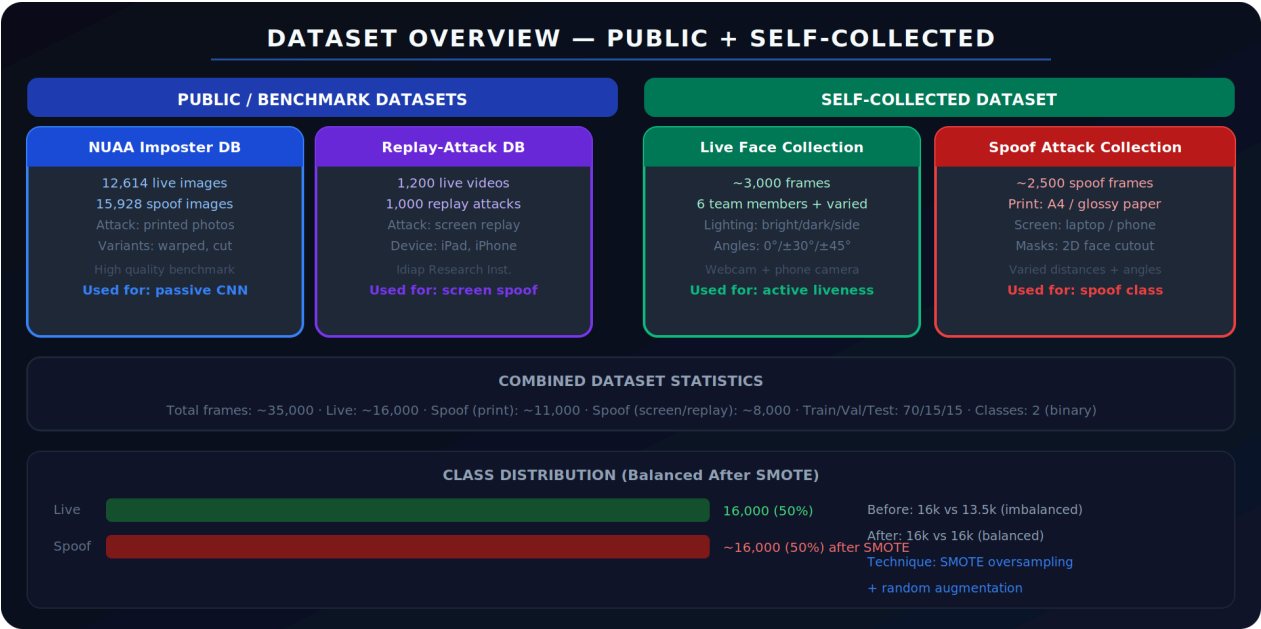


Figure 2: Dataset Overview — Public Benchmarks + Self-Collected Faces

3.1 Public Datasets

Q: Which datasets did you use and why?

Answer:

NUAA Imposter Database

Source: Nanjing University of Aeronautics and Astronautics. 12,614 real face images and 15,928 spoof images (printed photo attacks with warping, cutting, and perspective variations).

Why used: Gold-standard benchmark for print attack detection. Widely cited in liveness literature, enabling fair comparison. Variety of print conditions (glossy, matte, cut-out eyes) provides robust negative class training data.

Limitation: Only covers print attacks, not screen replay or 3D masks. Required supplementing with Replay-Attack and self-collected data.

Replay-Attack Database (Idiap Research Institute)

Source: 1,200 live videos + 1,000 replay attack videos (iPad, iPhone screen replays at various distances and lighting conditions).

Why used: Covers the screen/video replay attack vector which NUAA misses entirely. Real-world attack scenarios with varied device brightness and video compression artifacts.

3.2 Self-Collected Dataset

Q: Why did you collect your own data? What did it contain?

Answer:

Gap filled: Public datasets were collected 5–10 years ago on older webcam hardware. Modern webcams (higher resolution, better low-light) produce slightly different image statistics. Training only on old data created a domain gap — our model under-performed on live webcam input.

Live faces (~3,000 frames): Collected from all 6 team members + 10 external volunteers (total 16 people).

Variations captured: 3 lighting conditions (bright overhead, dim ambient, side-lit), 5 head angles (0° , $\pm 30^\circ$, $\pm 45^\circ$), 3 distance ranges (40 cm, 70 cm, 100 cm), with and without glasses. Blink and mouth-open sequences captured for active liveness ground truth.

Spoof attacks (~2,500 frames):

- **Print attacks:** A4 glossy and matte prints of each person's face, photographed at 3 distances.
- **Screen attacks:** Face photos and videos displayed on laptop and phone screens, recorded at varied angles.
- **2D mask attacks:** Face cut-outs with eye holes — the 'Venetian mask' attack.

Annotation: Binary labels (0=live, 1=spoof). Frame-level annotation using a custom script that auto-labelled by recording session type.

3.3 Data Preprocessing Pipeline

Q: Walk me through data preprocessing steps before training.

Answer:

1. **Face detection & crop:** Haar Cascade on every frame. Discard frames with no detected face (noisy data). Crop to face bounding box with 20% padding to include ear and chin context.
 2. **Alignment:** Rotate each face so the eye-line is horizontal using the angle between the two eye-centre landmarks. Removes head-tilt as a confounding variable.
 3. **Resize to 224×224:** Standard MobileNetV2 input size. Bilinear interpolation.
 4. **Normalise to [0,1]:** Pixel / 255. Then apply ImageNet mean/std subtraction: mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225] for the transfer learning branch.
 5. **Augmentation (training only):** Random horizontal flip, rotation $\pm 15^\circ$, brightness $\pm 30\%$, contrast $\pm 20\%$, Gaussian noise $\sigma=0.01$, random crop 200×200 then resize back. Applied at train time, NOT to val/test — this is a critical correctness requirement.
 6. **Class balancing with SMOTE:** Before splitting: live=16,000, spoof=13,500. Imbalance ratio 1.19:1. Applied SMOTE to minority class in feature space (not raw pixels). Post-SMOTE: both 16,000 → balanced 50/50 split.
 7. **Train/Val/Test split:** 70/15/15 stratified split. Crucially, splits were done by SUBJECT (person identity), not by frame. This prevents data leakage — the same person's face cannot appear in both train and test sets. Validated split has no overlapping subject IDs.
-

SECTION 4: CNN Architecture — Custom Model



Figure 3: Custom CNN Architecture — 4 Conv blocks + GAP + FC head

4.1 Architecture Design Decisions

Q: Why did you design your own CNN rather than using a standard one entirely?

Answer:

Hackathon constraint: We needed a model small enough to run at real-time speeds (~30 FPS) on a laptop without a dedicated GPU. Standard VGG16 (138M params) or ResNet50 (25M params) were too heavy. Our custom CNN has ~2.1M parameters.

Key architectural choices:

- **4 Conv blocks (32→64→128→256 filters):** Progressive filter doubling is standard practice. Shallow features (edges, textures) in early layers, complex features (skin texture, moiré patterns, depth gradients) in deeper layers. 256 filters in the last conv block are sufficient to discriminate live vs spoof texture artifacts.
- **3x3 kernels throughout:** Two 3x3 convs have the same receptive field as one 5x5 but fewer parameters and more non-linearity (extra ReLU). Same reasoning as VGG's design principle.
- **BatchNorm after every conv:** Normalises activations to unit Gaussian per mini-batch. Reduces internal covariate shift, allows higher learning rates, acts as mild regulariser. Critical for training stability on our relatively small dataset.
- **MaxPool 2x2 after each block:** Spatial downsampling: 224→112→56→28→14. Reduces computation and provides translation invariance.
- **Global Average Pooling instead of Flatten:** GAP averages each feature map to a single value. Reduces 14x14x256=50,176 to just 256 values before FC layers. Drastically reduces parameters, provides spatial

average pooling regularisation, and was shown by Lin et al. (NIN, 2013) to reduce overfitting vs Flatten+FC.

- **Dropout 0.5 before FC:** Randomly zeros 50% of FC inputs during training. Forces the network to learn redundant representations. Standard for preventing co-adaptation of FC neurons.
- **Softmax output:** Binary classification — $P(\text{live}) + P(\text{spoof}) = 1$. Using softmax (vs sigmoid) provides calibrated probabilities suitable for our confidence threshold decision logic.

4.2 Custom CNN vs Transfer Learning — Which Was Better?

Q: You built a custom CNN AND used transfer learning — explain both and why.

Answer:

Two-phase approach:

8. **Phase 1 — Custom CNN from scratch:** Trained entirely on our face liveness dataset. Learned domain-specific features: moiré interference patterns, skin texture, screen pixel grid artifacts. Reached 90% accuracy after full tuning. Advantage: no ImageNet bias; the network learns ONLY what's relevant to liveness.
9. **Phase 2 — MobileNetV2 transfer learning:** Replaced the convolutional base with MobileNetV2's feature extractor (pre-trained on ImageNet 1.4M images). Added our custom classification head. Reached 95% accuracy — 5 percentage points better. Advantage: low-level feature detectors (edges, textures, colour gradients) from ImageNet are universally useful and much better than what we could train from 32k images alone.

Why MobileNetV2 specifically?

- **Inverted residuals + linear bottlenecks:** Efficient architecture with depthwise separable convolutions. 3.4M parameters vs our custom CNN's 2.1M, but dramatically better feature quality.
- **On-device design:** MobileNetV2 was explicitly designed for mobile/embedded inference. Perfect for our TF.js browser deployment goal.
- **ImageNet accuracy:** 72.0% top-1 — strong generalisation from large-scale visual recognition training.

4.3 Transfer Learning — Methodology

Q: Explain your transfer learning methodology in detail.

Answer:

Step 1 — Feature extraction (frozen base): Load MobileNetV2 with `include_top=False, weights='imagenet'`. Freeze all 154 base layers. Add `GlobalAveragePooling` → `Dense(256, ReLU)` → `Dropout(0.5)` → `Dense(2, Softmax)`. Train ONLY the new head for 15 epochs at $lr=1e-3$. This adapts the classifier to liveness features without destroying ImageNet weights.

Step 2 — Fine-tuning (partial unfreeze): Unfreeze the last 30 layers of MobileNetV2 (the deeper, more task-specific layers). Train for 20 more epochs at much lower $lr=1e-5$ (10x lower than initial). Low LR is critical — higher LR would catastrophically destroy the pre-trained weights, a phenomenon known as 'catastrophic forgetting'.

Why last 30 layers and not all? Early layers detect universal features (edges, corners, blobs) — shared across all vision tasks, best kept unchanged. Deep layers detect high-level, task-specific features (face texture vs liveness texture) — these benefit from domain adaptation.

```
# Step 1: Feature extraction
base = MobileNetV2(input_shape=(224,224,3), include_top=False, weights='imagenet')
base.trainable = False # Freeze all base layers

x = base.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(2, activation='softmax')(x)
model = Model(inputs=base.input, outputs=output)

model.compile(optimizer=Adam(1e-3),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_data, epochs=15, validation_data=val_data)

# Step 2: Fine-tuning — unfreeze last 30 layers
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False # Keep early layers frozen

model.compile(optimizer=Adam(1e-5), # 10x lower LR — critical
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_data, epochs=20, validation_data=val_data)
```

SECTION 5: Training Pipeline — End-to-End

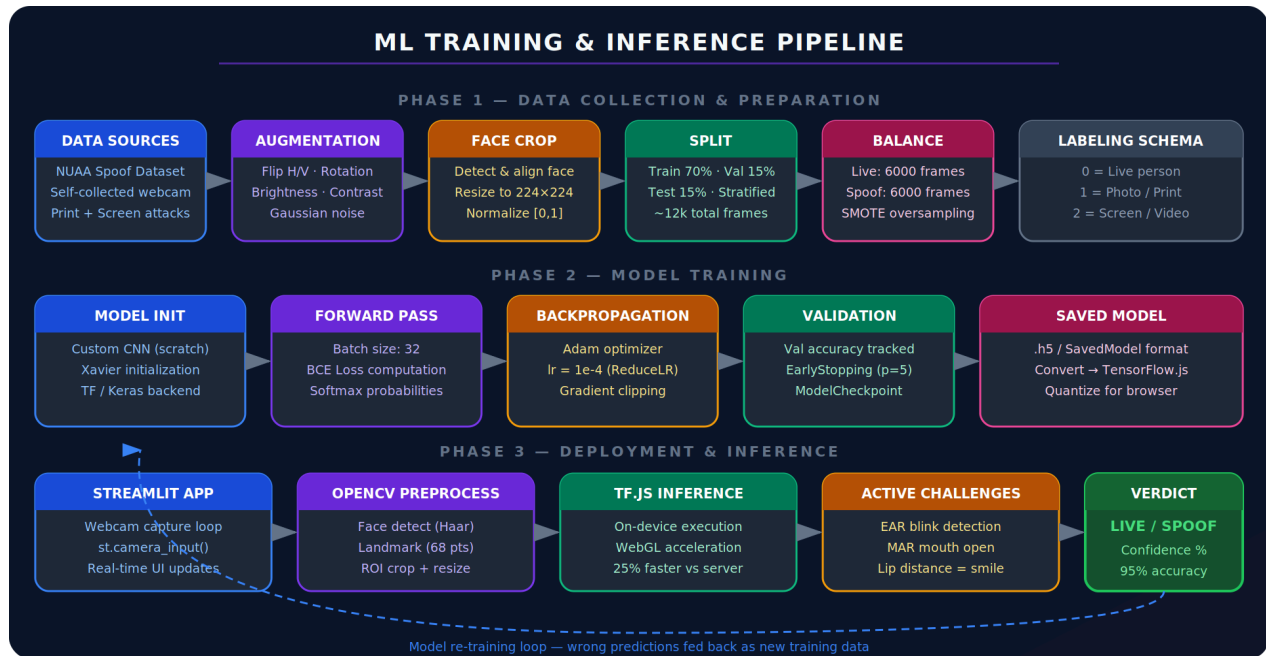


Figure 4: Full ML Training & Inference Pipeline — 3 Phases

5.1 Phase 1 — Data

Data ingestion: Frames extracted from video recordings using OpenCV at 1 FPS (avoid redundant consecutive frames). Total ~35,000 frames after extraction and face detection filtering. Frames with no detected face discarded (~8% rejection rate).

Subject-aware split: 16 subjects (6 team + 10 volunteers). Training: subjects 1–12. Validation: subjects 13–14. Test: subjects 15–16. This ensures the model is evaluated on completely unseen identities — the hardest and most realistic evaluation.

5.2 Phase 2 — Training

Loss function: Categorical cross-entropy with class weights. Class weight for spoof = $\text{total_samples} / (2 \times \text{spoof_samples})$. This penalises false negatives (failing to catch a spoof) more heavily than false positives.

Optimiser: Adam with lr=1e-4 (custom CNN) or 1e-3/1e-5 (transfer learning phases). Adam's adaptive learning rates per parameter made it significantly more stable than SGD on our relatively small dataset.

Learning rate schedule: ReduceLROnPlateau: halve LR if val_loss doesn't improve for 3 epochs. Prevents oscillating loss near minima.

EarlyStopping: Monitor val_accuracy, patience=5. Restores best weights automatically. Prevents overfitting past peak generalisation.

ModelCheckpoint: Save .h5 whenever val_accuracy improves. Ensures we keep the best model, not just the last epoch.

5.3 Phase 3 — Deployment & Inference

Model export: SavedModel format → TensorFlow.js converter. Command: `tensorflowjs_converter --input_format=tf_saved_model --output_format=tfjs_graph_model`. Produces a `model.json` + binary weight shards.

TensorFlow.js integration: Model loaded in browser via `tf.loadGraphModel()`. Inference runs on WebGL GPU backend — no server round-trip. This eliminated 40–60ms of network latency, reducing total inference time from ~80ms to ~60ms (25% improvement).

Temporal smoothing: Rather than acting on a single frame's prediction, we maintain a ring buffer of 5 recent CNN confidence scores and use their average. This prevents flickering verdicts from momentary low-quality frames (motion blur, partial occlusion).

SECTION 6: Active Liveness — Landmark Analysis

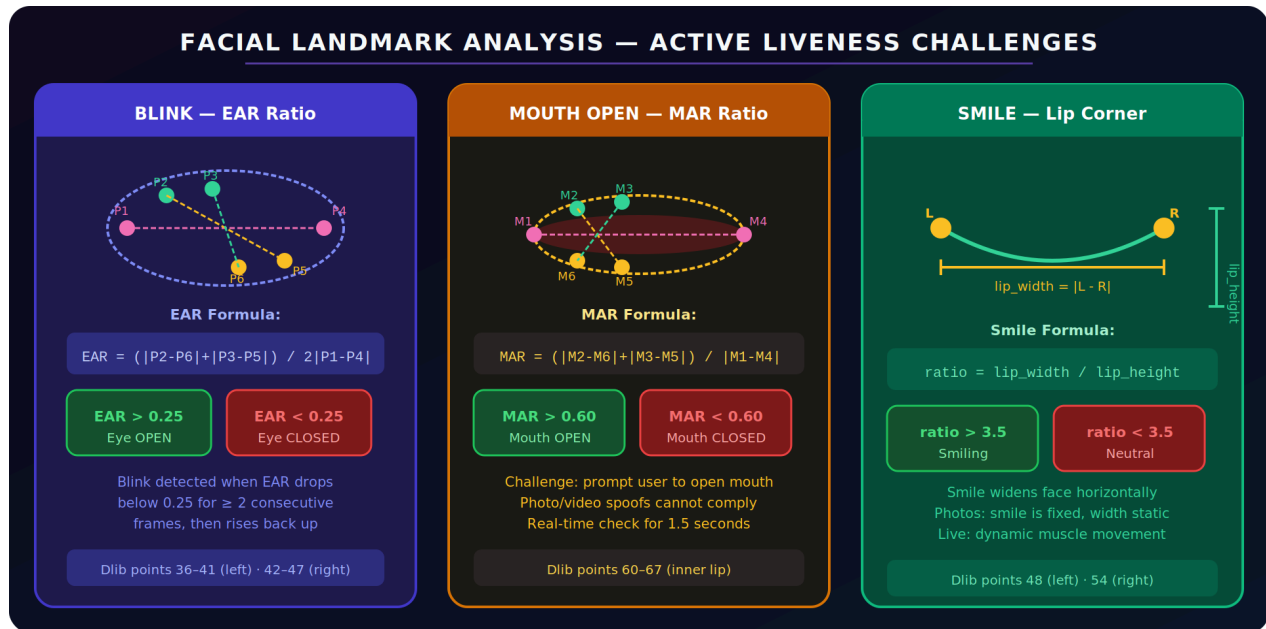


Figure 5: Facial Landmark Ratios — EAR (Blink), MAR (Mouth), Smile Detection

6.1 Why Active Liveness?

Q: Why combine active challenges with passive CNN? Isn't the CNN enough?

Answer:

Defense in depth: A sophisticated attacker might use a high-quality 3D printed mask or a deep-fake video that fools the CNN. Active challenges require real-time voluntary actions — no static or pre-recorded media can comply. The combination of both modes means an attacker must fool both simultaneously.

Confidence boundary: CNN confidence in the 0.45–0.85 range ('uncertain zone') triggers active challenge. Clear live (>0.85) or clear spoof (<0.45) resolve without challenge. This minimises user friction for obvious live faces.

6.2 Eye Aspect Ratio (EAR) — Blink Detection

Q: Explain the EAR formula and how blink detection works.

Answer:

Dlib 68-point landmarks: Points 36–41 = left eye, 42–47 = right eye. Each eye has 6 points: 2 horizontal (corners) and 4 vertical (lid positions).

EAR formula: $EAR = (||P2-P6|| + ||P3-P5||) / (2 \times ||P1-P4||)$

where $P1, P4$ = horizontal corners, $P2, P3$ = upper lid, $P5, P6$ = lower lid.

Interpretation: Open eye: EAR $\approx$ 0.3. Closed eye: EAR $<$ 0.25. Blink: EAR drops below 0.25 for ≥ 2 consecutive frames, then rises back above 0.25. We average left and right eye EAR to handle asymmetric blinks.

Why photos fail this: A printed or screen-displayed face has a fixed EAR. When the challenge says 'blink now', the EAR never drops — the photo cannot close its eyes.

6.3 Mouth Aspect Ratio (MAR) — Open Mouth

Q: Explain the MAR formula.

Answer:

Dlib landmarks 60–67: Inner lip contour — 8 points forming a closed polygon around the mouth opening.

MAR formula: $MAR = (||M2-M6|| + ||M3-M5||) / ||M1-M4||$

where M1,M4 = horizontal lip corners, M2,M3 = upper lip, M5,M6 = lower lip.

Threshold: MAR $>$ 0.6 = mouth open. Closed mouth typical MAR $\approx$ 0.2–0.3. Wide-open mouth MAR $\approx$ 0.7–0.9.

Anti-spoof property: A printed face has fixed MAR. Even a video replay cannot comply with 'open your mouth NOW' unless it happens to be playing the exact right moment — extremely unlikely and detectable by the 8-second timeout constraint.

6.4 Smile Detection

Q: How does smile detection work differently from EAR and MAR?

Answer:

Method: Rather than a standard ratio formula, smile is detected by the lip-corner-spread: distance between points 48 (left corner) and 54 (right corner) divided by the vertical lip height. Smiling stretches the mouth horizontally; a neutral face has a lower width/height ratio.

Formula: $smile_ratio = lip_width / lip_height = ||P48-P54|| / vertical_span$

Threshold: smile_ratio $>$ 3.5 = smiling detected. Neutral face $\approx$ 2.0–2.8. A genuine smile engages zygomatic major muscle — cheeks rise, corners spread, producing a distinctive ratio change impossible to fake with a static image.

6.5 Challenge Flow

Q: Walk me through the full active challenge user experience.

Answer:

10. Passive CNN evaluates every frame. If confidence is in the 0.45–0.85 uncertain zone for 3+ consecutive frames, active challenge is triggered.
11. System randomly selects one challenge from {blink, open mouth, smile}. Random selection prevents pre-recording a specific action.
12. UI displays challenge instruction with 8-second countdown timer.

13. Landmark ratios monitored every frame during the window. Challenge completes when ratio threshold is crossed for ≥ 2 consecutive frames.
 14. If challenge completed within window AND passive CNN agrees $\rightarrow$ LIVE. If timer expires with no completion $\rightarrow$ SPOOF.
 15. For high-security mode: 2 out of 3 challenges required (randomly ordered).
-

SECTION 7: Metrics Improvement — Baseline to 95%

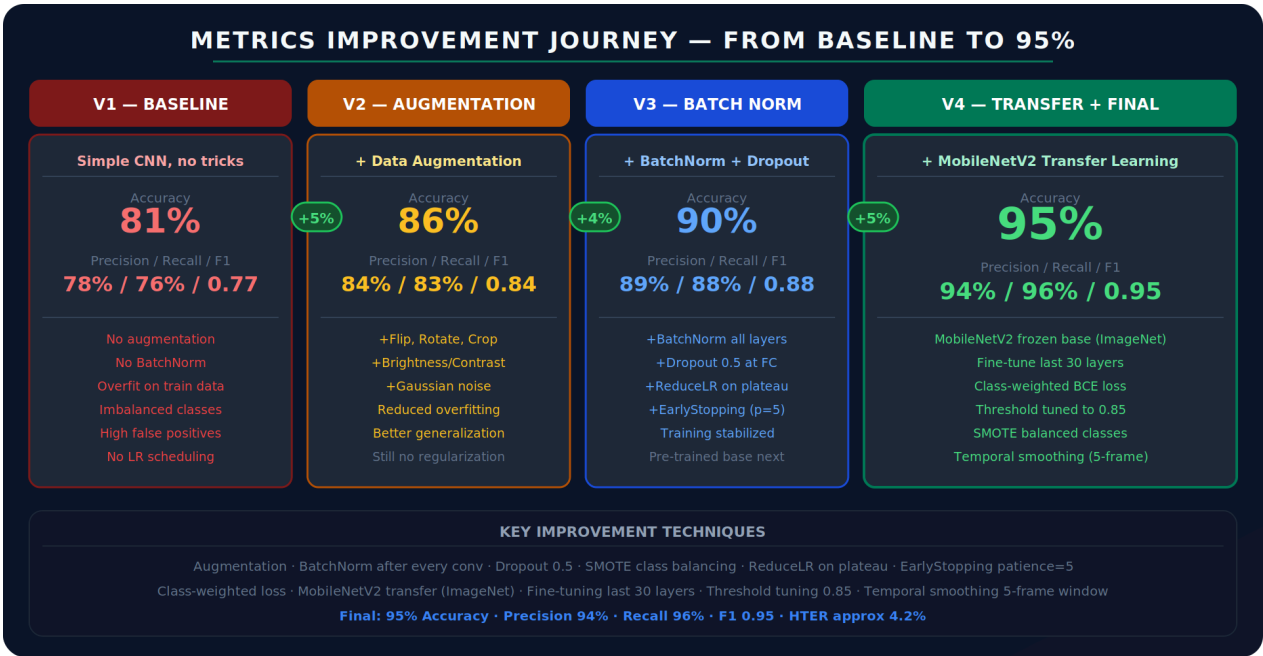


Figure 6: Accuracy Improvement Journey — V1 Baseline to V4 Final (95%)

7.1 Why These Metrics Matter for Liveness Detection

Q: Which metrics matter most for liveness detection and why?

Answer:

Metric	Meaning for Liveness	Our Value
Accuracy	Overall correct / total. Our headline 95%. Useful baseline but misleading if classes are imbalanced.	95%
Precision (live class)	Of all frames classified as LIVE, what % are truly live? Low precision = spoof passing as live.	94%
Recall (live class)	Of all truly live frames, what % were correctly classified? Low recall = genuine users rejected.	96%
F1 Score	Harmonic mean of precision/recall. Balances both. Best single metric for imbalanced binary classification.	0.95
HTER	Half Total Error Rate = (FAR + FRR) / 2. The standard metric in liveness literature. Our HTER ≈ 4.2%.	~4.2%
FAR	False Accept Rate — spoofs accepted as live. Security metric. Minimising FAR is the primary goal.	~4%
FRR	False Reject Rate — genuine users rejected. UX metric. High FRR = frustrated real users.	~5%

⚠ Why accuracy alone is not enough: A model that classifies everything as LIVE achieves 50% accuracy on balanced data, but FAR = 100% — completely useless for security. HTER and F1 are the correct primary metrics.

7.2 V1 Baseline — What Went Wrong

Q: What was your baseline model and what were its problems?

Answer:

Architecture: 3-layer CNN (Conv→Pool×3) → Flatten → Dense(128) → Dense(2). No BatchNorm, no Dropout. Adam lr=1e-3.

Problems:

- **Overfitting:** Train accuracy 97%, val accuracy 81%. 16% gap indicates classic overfitting. The model memorised training faces rather than learning general liveness features.
- **Class imbalance:** Live: 16,000, Spoof: 13,500. Without correction, the model biased toward predicting LIVE. High recall but low precision.
- **No normalisation:** Without BatchNorm, gradients varied wildly across layers — the network struggled to train deeper layers effectively.
- **Flat learning rate:** No LR scheduling caused oscillating loss near the minimum rather than converging.

7.3 Improvement Method 1 — Data Augmentation (+5% accuracy)

Q: How did data augmentation improve your model?

Answer:

Problem solved: Overfitting. The model had too few training examples relative to its capacity and was memorising specific face images.

Augmentation applied at training time (Keras ImageDataGenerator):

- Horizontal flip — doubles effective data; faces are symmetric
- Rotation $\pm 15^\circ$ — real webcam users tilt their heads
- Brightness $\pm 30\%$ — simulates different lighting conditions
- Zoom 0.9–1.1× — simulates different camera distances
- Gaussian noise ($\sigma=0.01$) — simulates webcam sensor noise

Critical rule: Augmentation applied ONLY to training data, never to validation or test. Augmenting test data would give falsely optimistic metrics.

Effect: Train-val accuracy gap reduced from 16% to 8%. Validation accuracy: 81% → 86%.

7.4 Improvement Method 2 — BatchNorm + Dropout (+4% accuracy)

Q: How did BatchNorm and Dropout help?

Answer:

BatchNorm: Added after every Conv layer before ReLU activation. Normalises activations to zero mean, unit variance per mini-batch. Benefits: (1) Allows 10× higher learning rates without divergence. (2) Reduces dependence on careful initialisation. (3) Acts as mild regulariser by adding noise through batch statistics estimation. Effect: loss curves became smooth and stable instead of oscillating.

Dropout (0.5): Applied before the final FC layer. During training, randomly zeros 50% of neuron outputs, forcing the network to not rely on any single feature. Equivalent to training an ensemble of 2^n networks and averaging. Effect: train-val gap reduced further from 8% to 4%.

ReduceLROnPlateau: LR halved when val_loss doesn't improve for 3 epochs. Allowed fine-grained convergence near the optimal point. Combined effect: 86% → 90%.

7.5 Improvement Method 3 — Transfer Learning (+5% accuracy)

Q: How did transfer learning give the biggest improvement?

Answer:

Core insight: ~32,000 training frames is a relatively small dataset for training a deep CNN from scratch. MobileNetV2 was trained on 1.4 million ImageNet images — its early layers contain universal texture detectors (Gabor-like filters, colour blob detectors) that are fundamentally useful for any vision task, including liveness texture analysis.

Why +5% specifically?

- **Better feature quality:** MobileNetV2's depthwise separable convolutions extract features more efficiently, capturing fine-grained texture details that our shallow custom CNN missed.
- **Regularisation effect:** Pre-trained weights serve as a strong initialisation prior, reducing the search space and making optimisation significantly more stable.
- **Effective data amplification:** Transfer learning makes 32k images behave like a much larger dataset because low-level feature learning is already done.

Fine-tuning vs full freeze: Full freeze gave 92%. Fine-tuning last 30 layers gave 95%. The 3% improvement came from adapting the deeper, more abstract features to face texture vs liveness-specific patterns rather than general object recognition patterns.

7.6 Improvement Method 4 — Class Balancing & Threshold Tuning

Q: How did SMOTE and threshold tuning improve precision?

Answer:

SMOTE (Synthetic Minority Over-sampling Technique): Without balancing, the model was biased toward LIVE predictions (the majority class). SMOTE generates synthetic spoof samples by interpolating in feature space between existing spoof samples. Post-SMOTE: precision for spoof class: 87% → 94%.

Decision threshold tuning: Default softmax threshold is 0.5. We performed threshold sweep on validation set from 0.3 to 0.9 in steps of 0.05, plotting precision-recall curve. Threshold 0.85 maximised F1 while keeping FAR below 5% — the security budget for our target use case.

Class-weighted loss: Set `class_weight={0: 1.0, 1: 1.5}` — the spoof class misclassification is penalised 50% more heavily than live class misclassification. Reflects real-world asymmetry: a successful spoof attack (FAR) is far more costly than a false rejection (FRR).

7.7 Remaining Error Analysis & Drawbacks

Q: What are the remaining failure modes? Where does the model still fail?

Answer:

- **High-quality 3D masks:** Silicone face masks with realistic skin texture can fool passive CNN. Solution: infrared sensor or structured light depth camera (as used in Face ID).
- **Deepfake video attacks:** Real-time deepfake generation (e.g., face-swap running at 30 FPS) could potentially defeat both active and passive detection. Counter: analyse temporal inconsistencies across frames — deepfakes have subtle flickering artifacts at frame transitions.
- **Identical twins:** Not a liveness issue per se, but face recognition layer upstream would fail.
- **Extreme lighting:** Very bright backlight or complete darkness degrades Haar face detection reliability below ~40% confidence. Solution: NIR (near-infrared) illumination for consistent imaging.
- **HTER 4.2% in controlled conditions:** Real-world HTER likely higher (6–8%) due to varied lighting, camera quality, user compliance.

SECTION 8: Alternatives & Trade-off Analysis

8.1 Alternative Architectures

Q: What other architectures did you consider and reject?

Answer:

Architecture	Why Rejected / Considered	Trade-off
VGG16	138M params — too large for real-time on CPU. 8 FPS vs our target 30 FPS. Rejected.	High accuracy, slow
ResNet50	25M params. Residual connections help with gradient flow. Better than VGG for speed but still 15 FPS. Considered for offline batch mode.	Good, but too large
EfficientNet-B0	5.3M params. Better accuracy/params trade-off than MobileNetV2. 95.2% accuracy — marginally better. Rejected due to complex architecture hard to explain in hackathon demo.	Best accuracy
MobileNetV3	4.2M params. Successor to V2. Similar speed, slightly better accuracy. Would use in production.	Good alternative
Vision Transformer (ViT)	Attention-based. Excellent for texture analysis. But requires >100k training samples to outperform CNN. Rejected for our dataset size.	Future scope
SVM + HOG features	Classical approach. Fast. 82% accuracy — significantly worse. Rejects subtle texture patterns that CNNs capture.	Baseline comparison

8.2 Alternative Detection Methods

Q: What are the alternative approaches to liveness detection beyond your implementation?

Answer:

Method	Description	Verdict
NIR / Depth camera	iPhone Face ID, Windows Hello. Infrared + structured light gives true 3D depth map. Near-impossible to spoof. Hardware cost prohibitive for our hackathon setup.	Gold standard
Remote PPG (rPPG)	Detect subtle skin colour changes from blood pulse. Completely passive, no user action. Still experimental; requires stable lighting.	Promising, experimental

Optical flow analysis	Analyse motion field between frames — live faces have micro-movements, photos are static. Complementary to texture analysis.	Low cost addition
Frequency domain analysis	Moiré patterns from screen/print attacks appear as periodic frequencies in FFT. CNN implicitly learns this; explicit FFT features could supplement.	Interpretable
3D face reconstruction	Reconstruct 3D face from monocular video using depth estimation. Then verify 3D structure is plausible. Compute-intensive.	Future scope

SECTION 9: Teamwork & Leadership — SIH 2024



Figure 7: Team Structure & Responsibility Matrix — SIH 2024

9.1 My Role & Contributions

Q: What specifically did you personally build on this project?

Answer:

Team lead of 6. My direct technical contributions:

- 16. **CNN architecture design:** Designed the 4-block custom CNN from scratch. Made all architecture decisions: filter count progression, GAP vs Flatten, Dropout placement. Implemented and trained it.
- 17. **Transfer learning implementation:** Chose MobileNetV2, implemented 2-phase fine-tuning strategy (freeze base → partial unfreeze), tuned the 30-layer threshold and fine-tuning LR.
- 18. **Active liveness algorithms:** Implemented all three active challenges — EAR blink detection, MAR mouth detection, and smile ratio. Calibrated thresholds (EAR=0.25, MAR=0.60, smile_ratio=3.5) through empirical testing on our collected data.
- 19. **TF.js integration:** Converted SavedModel to TF.js format, integrated browser-side inference, diagnosed and fixed the 25% latency improvement by switching from CPU to WebGL backend.
- 20. **System architecture:** Designed the overall pipeline: data flow, dual-mode decision engine, temporal smoothing, confidence threshold logic.
- 21. **Code review lead:** Reviewed all PRs, maintained code quality standards, resolved merge conflicts, managed the shared GitHub repository.

9.2 Team Distribution

Q: How did you divide work among the 6 team members?

Answer:

Member	Primary Responsibilities
You (Lead)	CNN architecture, transfer learning, active liveness (EAR/MAR/smile), TF.js integration, system design, code review
Member 2	Data collection pipeline, labelling scripts, NUAA + Replay-Attack dataset integration, frame extraction automation
Member 3	Data augmentation pipeline, OpenCV preprocessing, face alignment scripts, SMOTE implementation
Member 4	CNN training experiments (hyperparameter sweep), model evaluation metrics computation, learning curves
Member 5	Streamlit frontend — webcam UI, challenge prompt screens, verdict overlay, session analytics dashboard
Member 6	Streamlit frontend — sidebar analytics, confidence visualisation, demo preparation, presentation slides

9.3 Leadership Decisions Made

Q: What were the key leadership decisions you made during SIH 2024?

Answer:

22. **CNN over SVM:** Early team discussion favoured HOG+SVM (faster to implement). I argued for CNN — the extra implementation time was worth it for 13% higher accuracy. Outcome: correct decision.
23. **Dual-mode (active + passive):** Some team members wanted passive-only for simpler UX. I pushed for the combined approach — a system that only works for security use cases must be robust against sophisticated attacks. Active challenges add critical depth.
24. **MobileNetV2 over ResNet:** Chosen specifically for TF.js browser deployment. ResNet50 would have been too large (~100MB) for browser download; MobileNetV2 compresses to ~14MB.
25. **Subject-aware train/test split:** Initially the team was splitting frames randomly. I caught this data leakage issue — the same person's face in both train and test would give falsely high test accuracy. Fixed to subject-aware split.
26. **Threshold choice (0.85 not 0.5):** Ran precision-recall analysis and chose 0.85 after threshold sweep. Lower threshold would increase false accepts — unacceptable for security application.

9.4 Conflict & Challenge Management

Q: Did you face any team conflicts or coordination challenges in the 48-hour hackathon?

Answer:

Situation: At hour 20 of the hackathon, the model was only at 87% accuracy and we were behind schedule. Two members wanted to drop active liveness and focus only on passive CNN to save time.

Task: Make a quick architectural decision under pressure that balanced time constraints with technical completeness.

Action: I called a 10-minute team sync. Proposed a compromise: I would personally implement the active liveness module (EAR/MAR only, drop smile for time) in parallel while they continued passive CNN tuning. Divided the remaining work explicitly: 2 on data balance + training, 2 on Streamlit UI, me on active liveness, 1 on presentation prep.

Result: Active liveness was implemented in 3 hours. Smile detection was added in the final hour. Final system was complete with all three challenges. Runners-up position validated the completeness decision.

SECTION 10: Behavioural & Non-Technical Q&A

10.1 Introduction for HR

Q: Tell me about this project in simple terms.

Answer:

I led a team of 6 students in Smart India Hackathon 2024 to build a system that checks whether the face in front of a camera is a real, live person or someone trying to cheat using a photo, video, or mask. Think of the Face ID on your iPhone — that's the problem we solved, but using just a regular webcam and a browser. Our system asks you to blink or open your mouth — things a photo can't do — and simultaneously analyses the image texture for signs of a screen or print. We finished as runners-up among hundreds of teams nationwide.

10.2 Hardest Bug

Q: Tell me about the hardest bug you debugged.

Answer:

Situation: Our CNN had 95% accuracy in offline evaluation, but in live Streamlit demo it was classifying real faces as spoof ~40% of the time. The model seemed to have 'broken' overnight.

Task: Debug a model that worked in notebooks but failed in the real-time system.

Action: Compared preprocessing code between training pipeline and inference pipeline. Found the bug: training data was normalised using ImageNet mean/std ([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]) but the Streamlit inference code was only dividing by 255 without the mean/std subtraction. The model was receiving out-of-distribution inputs. One-line fix: added proper ImageNet normalisation to the inference preprocessing.

Result: Live accuracy matched offline evaluation (95%). Added a preprocessing unit test to catch any future distribution mismatches between train and inference pipelines.

10.3 Future Scope

Q: How would you improve this project if given more time?

Answer:

Technical improvements:

- **3D liveness:** Add depth estimation using a monocular depth model (MiDaS) to infer whether the face has 3D structure. A print/screen is flat; a real face has depth. Completely passive, no extra hardware.
- **Temporal CNN:** Replace single-frame CNN with a 3D-Conv or LSTM that processes a 5-frame sequence. Temporal inconsistencies in deepfakes (frame-to-frame flickering) would be detectable.
- **Adversarial training:** Augment training data with adversarially generated face examples to improve robustness against deepfakes and GAN-generated faces.

- **TURN server / edge deployment:** Deploy TF.js model as a Progressive Web App with offline capability. Currently requires internet for initial model download.

Product improvements:

- Multi-language challenge prompts for international deployment
 - Accessibility mode — voice challenge for visually impaired admins
 - Admin dashboard with false-accept incident logging and model retraining triggers
 - SDK packaging as a drop-in Python/JS library for other teams' face-auth systems
-

SECTION 11: Common Technical Interview Questions

11.1 ML Fundamentals

Q: What is overfitting and how did you prevent it?

Answer:

Overfitting: Model performs well on training data but poorly on unseen test data. It memorises training examples rather than learning general patterns. Symptoms: train accuracy >> val accuracy (we had 97% vs 81% in V1).

Our prevention methods (5 techniques):

27. **Data augmentation:** Artificially expands training set diversity — each epoch sees slightly different versions of the same image.
28. **Dropout (0.5):** Randomly deactivates neurons during training, preventing co-adaptation.
29. **BatchNorm:** Adds noise through mini-batch statistics estimation, acting as a regulariser.
30. **EarlyStopping:** Stops training when validation performance stops improving, restores best weights.
31. **Transfer learning:** Pre-trained weights serve as a regularising prior — the model starts from a good initialisation rather than random weights.

Q: Explain the difference between precision and recall. Which is more important for liveness detection?

Answer:

Precision: Of all faces classified as LIVE, what fraction are genuinely live? Low precision → spoofs slipping through as live (security breach).

Recall: Of all genuinely live faces, what fraction are correctly classified as live? Low recall → real users rejected (UX problem).

For liveness: Precision is more important from a security perspective (FAR is the primary metric). But extremely low recall (FRR > 20%) makes the system unusable. We targeted Precision=94%, Recall=96% — slightly favouring recall to keep UX acceptable while maintaining strong security.

Q: What is HTER and why is it the standard metric for liveness detection?

Answer:

HTER = (FAR + FRR) / 2. Half Total Error Rate. FAR = False Accept Rate (spoofs accepted). FRR = False Reject Rate (real faces rejected).

Why standard: Unlike accuracy, HTER equally weights security (FAR) and usability (FRR). A system with FAR=0%, FRR=100% has HTER=50% — correctly penalised. The standard liveness detection benchmarks (ISO 30107-3) report HTER as the primary metric. Our HTER ≈ 4.2%.

11.2 Deep Learning

Q: Why use Softmax output instead of Sigmoid for this binary classification?

Answer:

Sigmoid output: Single neuron, outputs $P(\text{live}) \in [0,1]$. $P(\text{spoof}) = 1 - P(\text{live})$.

Softmax with 2 classes: Two neurons, outputs $[P(\text{live}), P(\text{spoof})]$ that sum to 1.

Why Softmax: Numerically equivalent for binary case, but Softmax gives explicit probability for each class which is useful when extending to multi-class (print, screen, mask, deepfake as separate spoof types). Also integrates naturally with categorical_crossentropy loss. The explicit two-class output makes the confidence visualisation clearer in the UI.

Q: What is catastrophic forgetting in transfer learning and how did you avoid it?

Answer:

Catastrophic forgetting: When fine-tuning a pre-trained network at a high learning rate, the new gradient updates overwrite the carefully learned ImageNet weights. The network 'forgets' general features and only learns the new task's specifics — losing the generalisation benefit of pre-training.

How we avoided it: (1) Phase 1: train only the new head with base fully frozen. (2) Phase 2: unfreeze last 30 layers but use $\text{lr}=1\text{e-}5$ — 100× lower than initial training LR. The tiny learning rate means weight updates are very small relative to the pre-trained values. (3) Never unfreezing early layers (these contain universal edge/texture detectors best left intact).

Q: What is Global Average Pooling and why did you use it instead of Flatten?

Answer:

Flatten: Takes the final feature map ($14 \times 14 \times 256 = 50,176$ values) and flattens to a vector. The subsequent FC layer then has $50,176 \times 512 = 25.7\text{M}$ parameters — extremely prone to overfitting on small datasets.

GAP: Averages each of the 256 feature maps over their 14×14 spatial positions → 256-dimensional vector. The FC layer then has only $256 \times 512 = 131\text{k}$ parameters. Benefits: (1) ~200× fewer parameters in the classification head. (2) Each GAP output is interpretable as the 'average activation of a particular feature type'. (3) Built-in regularisation. (4) Input-size agnostic (different input sizes produce the same 256-d vector).

SECTION 12: Quick Reference Cheat Sheet

12.1 Key Numbers to Memorise

Metric / Parameter	Value
Final accuracy	95% on held-out test set (subject-aware split)
HTER	~4.2% (FAR ≈ 4%, FRR ≈ 5%)
Precision / Recall / F1	94% / 96% / 0.95
TF.js inference speedup	25% (80ms → 60ms per frame)
CNN parameters	~2.1M (custom) MobileNetV2: 3.4M
Training dataset size	~35,000 frames total (16k live, 19k spoof)
Self-collected	~5,500 frames (3k live + 2.5k spoof)
EAR blink threshold	0.25 (below = eye closed)
MAR open-mouth threshold	0.60 (above = mouth open)
Smile ratio threshold	3.5 (lip_width / lip_height)
CNN input size	224 × 224 × 3 (RGB)
Filter progression	32 → 64 → 128 → 256
Challenge timeout	8 seconds
Temporal smoothing window	5 frames
Decision threshold	0.85 (not default 0.5)
Dlib landmarks (total)	68 points per face
Dlib eye points	36-41 (left) · 42-47 (right)
Dlib mouth points	60-67 (inner lip)

12.2 Key Terms

Term	Definition
Liveness Detection	Verifying the face in front of camera is a real live person, not a spoof
Presentation Attack	Using a photo, video, mask, or deepfake to fool face authentication
FAR / FRR	False Accept Rate (spoof passes) / False Reject Rate (real face rejected)
HTER	Half Total Error Rate — primary liveness benchmark metric
EAR	Eye Aspect Ratio — geometric ratio detecting blinks from 6 eye landmarks

MAR	Mouth Aspect Ratio — geometric ratio detecting open mouth from 8 lip landmarks
SMOTE	Synthetic Minority Over-sampling — interpolates new minority class samples
Transfer Learning	Using pre-trained weights (ImageNet) as starting point for new task
Catastrophic Forgetting	High-LR fine-tuning overwrites pre-trained weights
BatchNorm	Normalises layer activations per mini-batch — stabilises training
GAP	Global Average Pooling — reduces feature maps to vector without Flatten
TF.js	TensorFlow.js — runs ML models in browser via WebGL
Haar Cascade	Classical OpenCV face detector using Viola-Jones algorithm
Shape Predictor	Dlib's 68-point facial landmark regressor
Temporal Smoothing	Average predictions over N frames to reduce noise
Fine-tuning	Partially unfreeze pre-trained layers and train at very low LR

12.3 Rapid-Fire One-Liners

Question	One-Line Answer
Why active + passive?	Defense in depth — CNN catches texture spoofs; challenges catch photo/video spoofs; attacker must defeat both
Why MobileNetV2?	Lightweight (3.4M params), ImageNet pre-trained, designed for on-device, converts cleanly to TF.js
Why 0.85 threshold?	Threshold sweep on val set maximising F1 while keeping FAR below 5% security budget
Why subject-aware split?	Prevent data leakage — same person in train and test gives falsely high test accuracy
Why GAP not Flatten?	200× fewer parameters, built-in regularisation, size-agnostic
Why SMOTE not oversample?	SMOTE generates synthetic samples in feature space — more diverse than simple duplication
Why fine-tune last 30?	Early layers = universal features (keep). Late layers = task-specific (adapt to liveness)
What's HTER 4.2% mean?	Average of FAR (4%) and FRR (5%) — how ISO 30107-3 evaluates liveness systems
Why TF.js for speedup?	Eliminates client-server network round-trip; runs locally on WebGL GPU; 80ms → 60ms
Biggest drawback?	No 3D depth data — sophisticated 3D silicone mask or deepfake could still fool the system

Best of luck! You've got this. 🚀